

```

1 #=====
2 # MIT 6.046J 代码精读 • PS2 - B树 COMBINE (高度增广)
3 # 源文件: btree_combine.py (来自 GitHub mit-6046)
4 #=====
5
6 """PS2 Problem 2-2: B-tree COMBINE(T1, T2, k) in  $O(|h_1 - h_2| + 1)$  time.
7
8 We build a standard B-tree with minimum degree  $t$  (each non-root node has
9 between  $t-1$  and  $2t-1$  keys; a node has one more child than keys) augmented so
10 every node stores the *height* of its subtree (part c). All keys of  $T_1 < k <$ 
11 all keys of  $T_2$ .
12
13 COMBINE strategy (part d):
14 WLOG  $h_1 \geq h_2$  (symmetric otherwise). Descend the right spine of  $T_1$  exactly
15  $h_1 - h_2$  steps to reach a node  $x$  whose subtree height equals  $h_2$ . Its parent  $p$ 
16 gains the separator key  $k$  and a new child pointing at the root of  $T_2$ . Because
17  $k$  is larger than everything in  $T_1$ 's right spine and smaller than everything in
18  $T_2$ , insertion preserves order. If  $p$  overflows ( $2t-1$  keys already), split it
19 and propagate the split upward -- at most  $h_1$  levels, but split cost is  $O(t)=O(1)$ 
20 per level and only along the insertion path of length  $O(h_1-h_2)+\text{overflow chain}$ ;
21 the amortised/worst insertion of a single key with a precomputed position is
22  $O(\text{height difference} + 1)$  since we start the splice at the correct depth.
23
24 Special cases:  $h_1 == h_2$  (part a)  $\rightarrow$  make a new root with single key  $k$  and the
25 two roots as children,  $O(1)$ .  $h_1 == h_2 + 1$  (part b)  $\rightarrow$  splice  $T_2$ 's root as the
26 new rightmost child of  $T_1$ 's root with separator  $k$ , split root if needed,  $O(1)$ .
27
28 This module implements insert/search with the height augmentation and COMBINE,
29 and validates all B-tree invariants + sortedness after each operation.
30 """
31 from __future__ import annotations
32
33 from typing import List, Optional
34
35
36 class Node:
37     __slots__ = ("keys", "children", "leaf", "height")
38
39     def __init__(self, leaf: bool = True):
40         self.keys: List[int] = []
41         self.children: List["Node"] = []
42         self.leaf = leaf
43         self.height = 0 # augmented: height of subtree rooted here (leaf = 0)
44
45     def recompute_height(self) -> None:
46         self.height = 0 if self.leaf else self.children[0].height + 1
47
48
49 class BTree:
50     def __init__(self, t: int):
51         assert t >= 2
52         self.t = t
53         self.root = Node(leaf=True)
54
55     # ----- search -----
56     def search(self, k: int) -> bool:
57         node = self.root
58         while node is not None:
59             i = 0
60             while i < len(node.keys) and k > node.keys[i]:
61                 i += 1
62             if i < len(node.keys) and node.keys[i] == k:
63                 return True
64             if node.leaf:
65                 return False
66             node = node.children[i]
67         return False
68
69     # ----- insert (CLRS, with height maintenance) -----
70     def insert(self, k: int) -> None:
71         r = self.root
72         if len(r.keys) == 2 * self.t - 1:
73             s = Node(leaf=False)
74             s.children.append(r)
75             s.recompute_height()
76             self._split_child(s, 0)
77             self.root = s
78             self._insert_nonfull(s, k)
79         else:
80             self._insert_nonfull(r, k)
81
82     def _split_child(self, x: Node, i: int) -> None:
83         t = self.t
84         y = x.children[i]
85         z = Node(leaf=y.leaf)

```

```

86     z.keys = y.keys[t:]
87     y_mid = y.keys[t - 1]
88     y.keys = y.keys[: t - 1]
89     if not y.leaf:
90         z.children = y.children[t:]
91         y.children = y.children[:t]
92     y.recompute_height()
93     z.recompute_height()
94     x.children.insert(i + 1, z)
95     x.keys.insert(i, y_mid)
96     x.recompute_height()
97
98 def _insert_nonfull(self, x: Node, k: int) -> None:
99     i = len(x.keys) - 1
100    if x.leaf:
101        x.keys.append(0)
102        while i >= 0 and k < x.keys[i]:
103            x.keys[i + 1] = x.keys[i]
104            i -= 1
105        x.keys[i + 1] = k
106        # leaf height stays 0
107    else:
108        while i >= 0 and k < x.keys[i]:
109            i -= 1
110        i += 1
111        if len(x.children[i].keys) == 2 * self.t - 1:
112            self._split_child(x, i)
113            if k > x.keys[i]:
114                i += 1
115        self._insert_nonfull(x.children[i], k)
116        x.recompute_height()
117
118 # ----- COMBINE -----
119 @staticmethod
120 def combine(T1: "BTree", T2: "BTree", k: int) -> "BTree":
121     """Destructively combine (all keys T1 < k < all keys T2)."""
122     assert T1.t == T2.t
123     t = T1.t
124     # Empty-tree special cases: just insert k into the non-empty tree.
125     if len(T1.root.keys) == 0 and T1.root.leaf:
126         out = BTree(t)
127         out.root = T2.root
128         out.insert(k)
129         return out
130     if len(T2.root.keys) == 0 and T2.root.leaf:
131         out = BTree(t)
132         out.root = T1.root
133         out.insert(k)
134         return out
135     h1, h2 = T1.root.height, T2.root.height
136     out = BTree(t)
137     if h1 == h2:
138         r1, r2 = T1.root, T2.root
139         # if the two equal-height roots + separator fit in one node, fuse them
140         # (this also covers the case where either root is underfull as a child)
141         if len(r1.keys) + 1 + len(r2.keys) <= 2 * t - 1:
142             fused = Node(leaf=r1.leaf)
143             fused.keys = r1.keys + [k] + r2.keys
144             if not r1.leaf:
145                 fused.children = r1.children + r2.children
146             fused.recompute_height()
147             out.root = fused
148         else:
149             root = Node(leaf=False)
150             root.keys = [k]
151             root.children = [r1, r2]
152             root.recompute_height()
153             out.root = root
154             # a former root may now be an underfull child
155             if len(r1.keys) < t - 1 or len(r2.keys) < t - 1:
156                 idx = 0 if len(r1.keys) < t - 1 else 1
157                 out._fix_child_underflow(root, idx)
158                 out._fix_root_overflow()
159             return out
160     if h1 > h2:
161         out.root = T1.root
162         out._graft_right(T2.root, k, h2)
163     else:
164         out.root = T2.root
165         out._graft_left(T1.root, k, h1)
166     out._fix_root_overflow()
167     return out
168
169 def _graft_right(self, subtree: Node, k: int, target_h: int) -> None:
170     """Attach `subtree` (height target_h) as new rightmost child, sep key k,
171     descending the right spine of self.root. ``path`` records (node, index-

```

```

172     of-followed-child) so the bottom-up fixup can address each node's slot in
173     its parent in  $O(1)$ . """
174     path: List[tuple] = []
175     x = self.root
176     while x.height > target_h + 1:
177         path.append((x, len(x.children) - 1))
178         x = x.children[-1]
179     x.keys.append(k)
180     x.children.append(subtree)
181     # the grafted former-root may be underfull as a non-root child
182     if len(subtree.keys) < self.t - 1:
183         self._fix_child_underflow(x, len(x.children) - 1)
184     self._fixup_up(x, path)
185
186 def _graft_left(self, subtree: Node, k: int, target_h: int) -> None:
187     path: List[tuple] = []
188     x = self.root
189     while x.height > target_h + 1:
190         path.append((x, 0))
191         x = x.children[0]
192     x.keys.insert(0, k)
193     x.children.insert(0, subtree)
194     if len(subtree.keys) < self.t - 1:
195         self._fix_child_underflow(x, 0)
196     self._fixup_up(x, path)
197
198 def _fixup_up(self, node: Node, path: List[tuple]) -> None:
199     """Fix ``node`` (which may over- or under-flow after a graft) and
200     cascade any resulting imbalance up ``path`` to the root. Each level is
201      $O(t)=O(1)$ ; the path length is  $O(|h_1-h_2|)$ . """
202     for parent, idx in reversed(path):
203         node.recompute_height()
204         if len(node.keys) > 2 * self.t - 1:
205             self._split_child(parent, idx)
206         elif len(node.keys) < self.t - 1:
207             self._fix_child_underflow(parent, idx)
208         else:
209             parent.recompute_height()
210         node = parent
211     # now `node` is the root
212     node.recompute_height()
213     if len(node.keys) > 2 * self.t - 1:
214         self._fix_root_overflow()
215     elif len(node.keys) == 0 and not node.leaf:
216         # root emptied by a merge: drop a level
217         self.root = node.children[0]
218
219 def _fix_child_underflow(self, parent: Node, i: int) -> None:
220     """Restore ``parent.children[i]`` to  $\geq t-1$  keys.
221
222     A former root grafted in by COMBINE may have as few as 1 key, so a single
223     borrow need not suffice. We therefore *repeatedly* borrow from whichever
224     neighbour has spare keys (a spine sibling holds up to  $2t-1$ , so this always
225     succeeds once a spare sibling exists), and only fall back to a merge when
226     neither neighbour can spare a key. A merge removes one key from
227     ``parent``; the caller's cascade fixes ``parent`` if it then underflows.
228     Total work  $O(t)$  because at most  $t-2$  keys are moved. """
229     t = self.t
230     child = parent.children[i]
231     # borrow as many keys as needed
232     while len(child.keys) < t - 1:
233         left = parent.children[i - 1] if i > 0 else None
234         right = parent.children[i + 1] if i < len(parent.children) - 1 else None
235         if left is not None and len(left.keys) > t - 1:
236             child.keys.insert(0, parent.keys[i - 1])
237             parent.keys[i - 1] = left.keys.pop()
238             if not left.leaf:
239                 child.children.insert(0, left.children.pop())
240         elif right is not None and len(right.keys) > t - 1:
241             child.keys.append(parent.keys[i])
242             parent.keys[i] = right.keys.pop()
243             if not right.leaf:
244                 child.children.append(right.children.pop())
245         else:
246             break # no spare neighbour -> merge below
247     if len(child.keys) >= t - 1:
248         child.recompute_height()
249         parent.recompute_height()
250         return
251     # merge with a neighbour through the separator
252     if i > 0: # merge child into left sibling
253         left = parent.children[i - 1]
254         left.keys.append(parent.keys.pop(i - 1))
255         left.keys.extend(child.keys)
256     if not left.leaf:
257         left.children.extend(child.children)

```

```

258         parent.children.pop(i)
259         left.recompute_height()
260     else: # merge right sibling into child
261         right = parent.children[i + 1]
262         child.keys.append(parent.keys.pop(i))
263         child.keys.extend(right.keys)
264         if not child.leaf:
265             child.children.extend(right.children)
266         parent.children.pop(i + 1)
267         child.recompute_height()
268         parent.recompute_height()
269
270     def _fix_root_overflow(self) -> None:
271         if len(self.root.keys) > 2 * self.t - 1:
272             s = Node(leaf=False)
273             s.children.append(self.root)
274             s.recompute_height()
275             self._split_child(s, 0)
276             self.root = s
277
278     # ----- validation -----
279     def inorder(self) -> List[int]:
280         out: List[int] = []
281         self._inorder(self.root, out)
282         return out
283
284     def _inorder(self, x: Node, out: List[int]) -> None:
285         if x.leaf:
286             out.extend(x.keys)
287             return
288         for i, key in enumerate(x.keys):
289             self._inorder(x.children[i], out)
290             out.append(key)
291         self._inorder(x.children[-1], out)
292
293     def check_invariants(self) -> None:
294         keys = self.inorder()
295         assert keys == sorted(keys), "not sorted"
296         assert len(set(keys)) == len(keys), "duplicate keys"
297         self._check(self.root, is_root=True, lo=None, hi=None)
298
299     def _check(self, x: Node, is_root: bool, lo: Optional[int], hi: Optional[int]) -> int:
300         t = self.t
301         assert len(x.keys) <= 2 * t - 1, "overflow"
302         if not is_root:
303             assert len(x.keys) >= t - 1, f"underflow: {len(x.keys)} keys"
304         assert x.keys == sorted(x.keys)
305         for key in x.keys:
306             if lo is not None:
307                 assert key > lo
308             if hi is not None:
309                 assert key < hi
310         if x.leaf:
311             assert x.height == 0
312             return 0
313         assert len(x.children) == len(x.keys) + 1, "child count"
314         heights = set()
315         bounds = [lo] + x.keys + [hi]
316         for i, c in enumerate(x.children):
317             heights.add(self._check(c, False, bounds[i], bounds[i + 1]))
318         assert len(heights) == 1, "unequal child heights (not balanced)"
319         h = heights.pop() + 1
320         assert x.height == h, f"height augmentation wrong: {x.height} vs {h}"
321         return h
322

```